CAMPUSFIX

A Role-Based College Complaint and Maintenance Management System

Final Project Report

Fourth-Year CSE Mini Project

Student Name: [Student Name]

Roll Number: [Roll Number]

Guide Name: [Faculty Guide Name]

Academic Session: 2026-27

Department of Computer Science and Engineering

Certificate and Declaration

This page is intentionally provided for institution-approved certificate and declaration text.

Certified that the project titled CampusFix: A Role-Based College Complaint and Maintenance Management System is the original work of [Student Name], Roll Number [Roll Number], completed under the guidance of [Faculty Guide Name] during the academic session 2026-27.

Student signature: _____ Guide signature: _____

Acknowledgement

I express my sincere gratitude to [Faculty Guide Name], the Department of Computer Science and Engineering, and all faculty members who supported this mini project. I also thank the students and maintenance personnel whose practical needs informed the role and workflow design. Any remaining errors are my responsibility.

Abstract

CampusFix is a local web application for registering, assigning, tracking, resolving, and reviewing college maintenance complaints. It replaces informal reporting through verbal messages or disconnected channels with a role-based workflow for Students, Maintenance Staff, and Administrators. The completed Django application provides secure authentication, complaint tickets, validated image evidence, controlled status transitions, permanent history, notifications, feedback, priority scoring, analytics, filtered reporting, CSV exports, and print views. The implementation uses Python, Django 6.1.1, SQLite, Bootstrap, Chart.js, and Pillow. Verification at the end of Batch 6 comprised 84 passing automated tests, a clean Django system check, no missing migrations, static-file collection, and live HTTP role checks. The project demonstrates how a small, maintainable monolithic architecture can provide accountability and operational visibility without requiring cloud services.

Table of Contents

1.1 Background 6
1.2 Problem Statement 7
1.3 Need for the System 7
1.4 Objectives 7
1.5 Scope 7
1.6 Limitations 7
2.1 Related Concepts 8
2.2 Comparative Review 8
2.3 Literature Review Findings 8
3.1 Functional Requirements 8
3.2 Non-functional Requirements 9
3.3 User Roles and Permissions 9
3.4 Use Cases 10
4.1 Overall Architecture 10
4.2 Role and Workflow Design 11
4.3 Database Design 11
4.4 Priority Scoring 12
4.5 Security and Privacy 12
5.1 Technology Stack 13
5.2 Authentication and Authorisation 13
5.3 Complaint Submission and Images 13
5.4 Workflow and Timeline 14
5.5 Notifications and Feedback 15

5.6 Analytics and Reporting 15
6.1 Testing Approach 18
6.2 Automated Evidence 18
6.3 Working Demonstration Evidence 19
6.4 Key Results 23
7.1 Conclusion 23
7.2 Future Scope 23

List of Figures

Figure 1. CampusFix architecture

Figure 2. Role-based permissions

Figure 3. Complaint lifecycle

Figure 4. Entity relationship overview

Figure 5. Priority scoring method

Figure 6. Public landing page

Figure 7. Student dashboard

Figure 8. Complaint submission form

Figure 9. Student ticket timeline

Figure 10. Maintenance Staff queue

Figure 11. Administrator workflow controls

Figure 12. Analytics dashboard

Figure 13. Reports and exports

Figure 14. Permission-denied page

List of Tables

Table 1. Project objectives

Table 2. Literature comparison

Table 3. Functional requirements

Table 4. Role permissions

Table 5. Data entities

Table 6. Technology stack

Table 7. Test evidence

Table 8. Important routes

Table 9. Priority scoring table

Chapter 1 Introduction

1.1 Background

College maintenance requests often begin as conversations, calls, or messages. These channels are easy to start but difficult to audit: reports may lack a location or evidence, assignments are unclear, and students

cannot see whether work has progressed. CampusFix models the full maintenance request as a permanent ticket shared through controlled role-specific views.

1.2 Problem Statement

The problem is the absence of one accountable process connecting the person who reports an issue, the administrator who reviews it, and the staff member who performs the work. A useful solution must preserve the ticket history, prevent unauthorised actions, communicate updates, and give administrators evidence for workload and service review.

1.3 Need for the System

- A unique ticket prevents a complaint from disappearing in informal communication.
- Role-specific queues clarify responsibility and reduce accidental disclosure.
- A permanent timeline records what changed, when, and by whom.
- Priority scoring makes urgency visible while keeping the rule understandable.
- Analytics, feedback, and reports turn operational records into review evidence.

1.4 Objectives

Objective	Implemented evidence
Central complaint record	Ticket number, category, location, description, urgency, optional evidence
Controlled responsibilities	Student, Maintenance Staff, and Administrator permissions
Traceable workflow	Seven statuses and permanent ComplaintHistory events
Timely awareness	Private in-app notifications and unread counts
Decision support	Priority score, analytics, reports, CSV and print output
Quality assurance	84 automated tests plus live HTTP checks

1.5 Scope

The accepted scope is a local, responsive college maintenance application. Public registration creates Students only. Administrators manage workflow and reports; Maintenance Staff act only on assigned work; Students see only their own complaints. The system is designed for demonstration and academic evaluation, not public production deployment.

1.6 Limitations

- SQLite and local media storage suit a single-machine demonstration, not a highly concurrent deployment.
- Notifications are in-app only; email and SMS are future work.
- Priority is deterministic and rule-based, not predictive machine learning.
- The project has no mobile application, multilingual interface, cloud hosting, or production monitoring.
- The print view intentionally limits browser rendering to 500 complaints; CSV exports include all matching records.

Chapter 2 Literature Review

2.1 Related Concepts

The design was informed by established ideas rather than copied from one product. NIST role-based access control associates permissions with roles instead of assigning each permission separately to each user [1]. Django provides authentication, authorisation, password handling, CSRF protection, parameterised ORM queries, and clickjacking middleware [2][3]. Jira describes work tracking through defined statuses and one-direction transitions [4]. SQLite provides a self-contained, serverless transactional database appropriate for a compact local application [5]. Chart.js renders browser charts but requires accessible names or text alternatives, which CampusFix supplements with tables [6]. OWASP documents formula injection when untrusted CSV cells begin with formula characters [7].

2.2 Comparative Review

Source / approach	Useful idea	CampusFix adaptation
NIST RBAC [1]	Permissions through assigned roles	Three explicit application roles with server-side checks
Django authentication [2]	Integrated users, sessions, passwords	Custom user model and role-specific dashboard router
Django security [3]	CSRF, query parameterisation, clickjacking defence	Middleware plus CSRF on every state-changing form
Jira workflows [4]	Statuses, transitions, assignee, auditability	Restricted complaint lifecycle and permanent history
SQLite [5]	Zero-configuration embedded SQL	Single local database for academic deployment
Chart.js [6]	Responsive canvas charts	Administrator charts with adjacent data tables
OWASP CSV Injection [7]	Spreadsheet formulas can execute untrusted text	Prefix formula-leading exported cells with an apostrophe
Bootstrap [8]	Responsive grid and accessible components	Consistent mobile-friendly interface

2.3 Literature Review Findings

The review suggests that a complaint system needs more than a submission form. Accountability depends on explicit roles, constrained transitions, a durable audit trail, and evidence that users can understand. CampusFix therefore combines a helpdesk-style ticket with role gates, a lifecycle service, private notifications, and reports. It remains intentionally smaller than enterprise service-management tools and is tailored to one college demonstration.

Chapter 3 Requirements and Analysis

3.1 Functional Requirements

ID	Requirement	Result

ID	Requirement	Result
FR-01	Student registration, login, logout, profile and password change	Implemented
FR-02	Submit complaint with category, location, urgency and optional image	Implemented
FR-03	Student-only ownership view with search and filters	Implemented
FR-04	Administrator review, assignment, reassignment, rejection and closure	Implemented
FR-05	Assigned Staff start and resolve workflow	Implemented
FR-06	Permanent timeline and private notifications	Implemented
FR-07	One feedback response for a resolved or closed complaint	Implemented
FR-08	Priority, analytics, reports, CSV and print views	Implemented

3.2 Non-functional Requirements

- Security: server-side role and ownership enforcement, CSRF protection, safe local redirects, password validation, and validated image content.
- Usability: responsive layout, readable status and priority indicators, clear empty states, keyboard focus visibility, and branded errors.
- Integrity: workflow transitions are centralised and history entries are permanent.
- Performance: paginated lists and related-object loading limit page size and database queries.
- Maintainability: one Django project with separate core, accounts, and complaints applications.
- Portability: pinned Python dependencies, SQLite, local media, and repeatable demo commands.

3.3 User Roles and Permissions

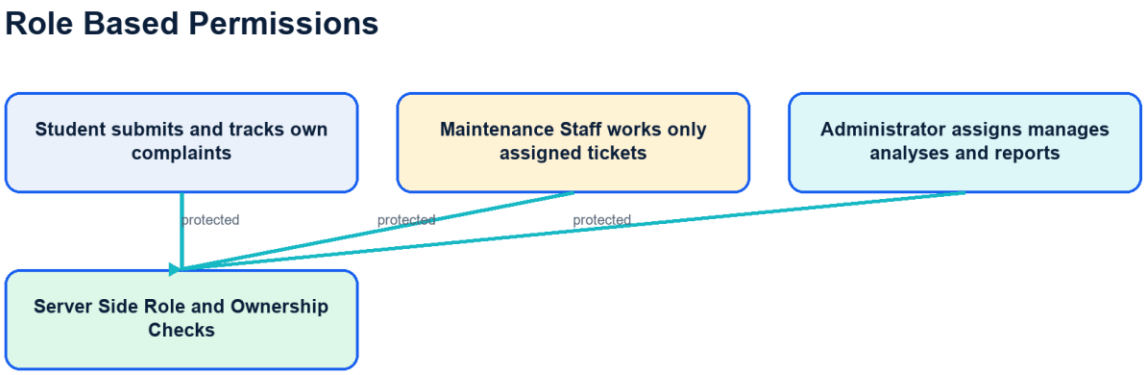


Figure 2. Role-based permissions

Capability	Student	Maintenance Staff	Administrator
Register publicly	Yes	No	No

Capability	Student	Maintenance Staff	Administrator
Create complaint	Yes	No	No
View complaint	Own only	Assigned only	All
Change workflow	No	Start / resolve assigned	Review / assign / reject / close
Submit feedback	Own completed ticket	No	No
Analytics and reports	No	No	Yes

3.4 Use Cases

Primary use cases are: create a Student account; submit and track a complaint; review and assign a complaint; begin and resolve assigned work; close or reject a complaint; receive a private notification; submit feedback; filter analytics and reports; export filtered CSV files; and print a filtered report.

Chapter 4 System Design

4.1 Overall Architecture

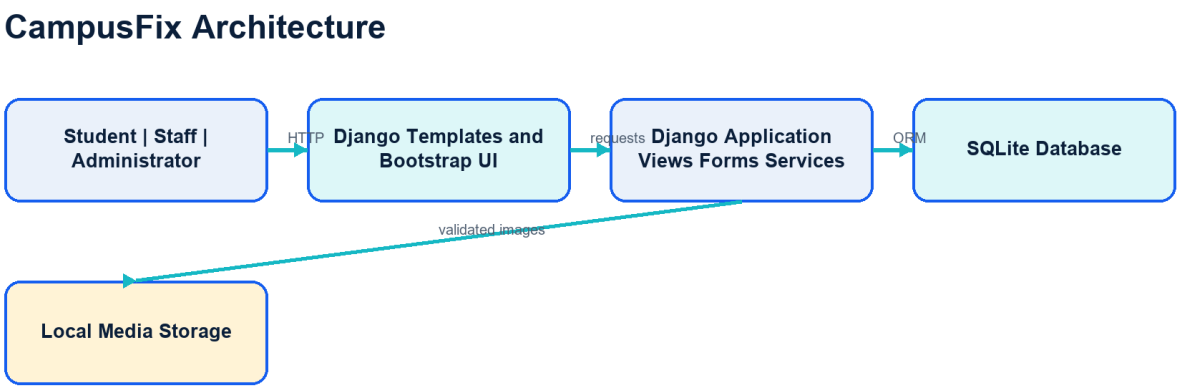


Figure 1. CampusFix architecture

The browser communicates with Django views rendered through templates. Forms validate input, decorators and service functions enforce access, the ORM stores structured records in SQLite, and validated evidence files are stored under the local media directory.

4.2 Role and Workflow Design

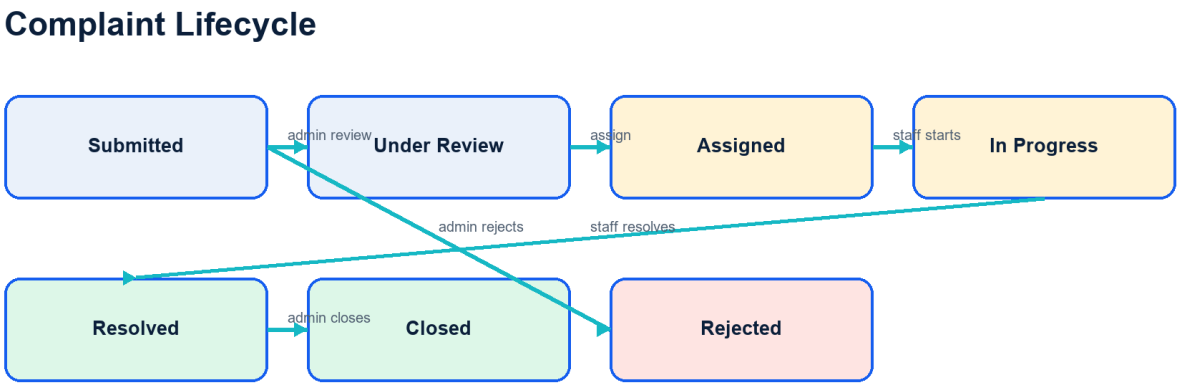


Figure 3. Complaint lifecycle

The normal path is Submitted to Under Review to Assigned to In Progress to Resolved to Closed. Administrators may assign directly from Submitted and may reject Submitted or Under Review complaints. Maintenance Staff cannot act on another staff member's assignment.

4.3 Database Design

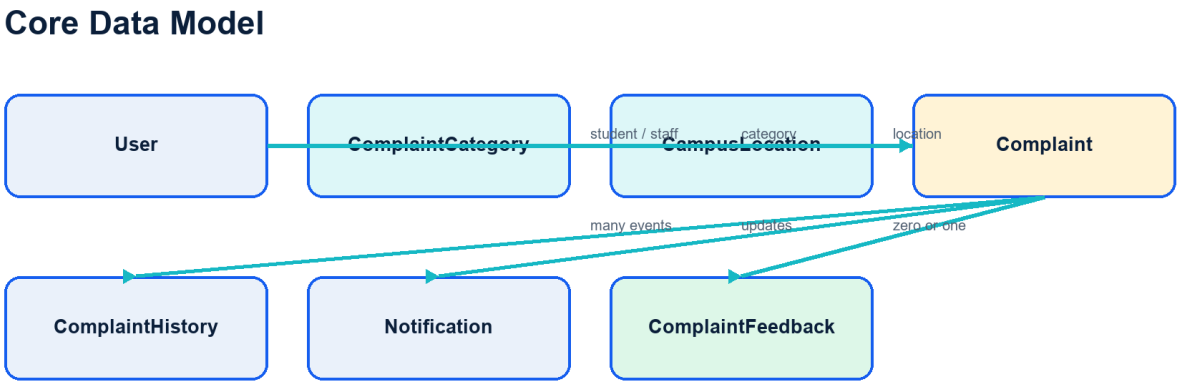


Figure 4. Entity relationship overview

Entity	Purpose	Key relationships

Entity	Purpose	Key relationships
User	Authentication profile and role	Creates complaints; receives assignments and notifications
ComplaintCategory	Classification reference data	One category to many complaints
CampusLocation	Campus place reference data	One location to many complaints
Complaint	Central ticket and workflow state	Student, category, location, optional assigned Staff
ComplaintHistory	Permanent workflow event	Many events per complaint; optional actor
Notification	Private account update	Recipient and optional complaint destination
ComplaintFeedback	Student rating and comment	One-to-one with completed complaint

4.4 Priority Scoring

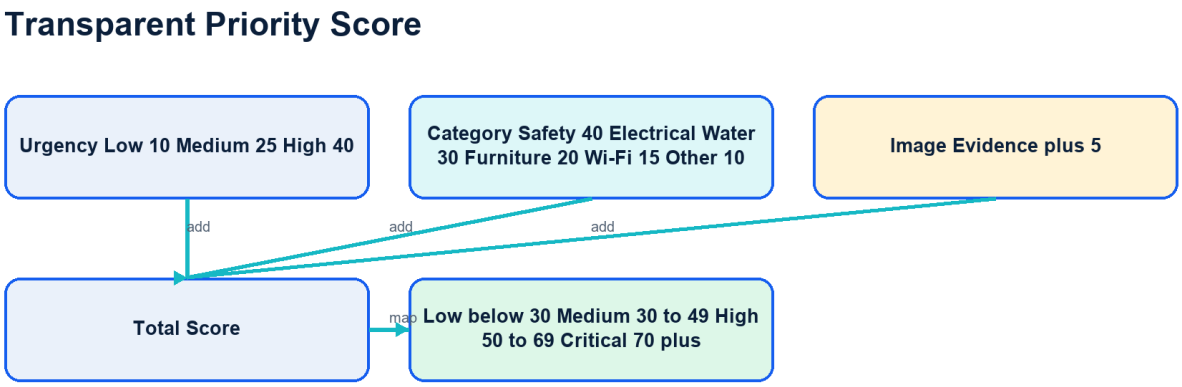


Figure 5. Priority scoring method

The exact score is urgency weight plus category weight plus five points when evidence is present. The explanation is saved with the complaint, making the decision inspectable instead of opaque.

4.5 Security and Privacy

- Every protected page checks authentication and the required CampusFix role on the server.
- Student ticket views verify ownership; Staff ticket views verify assignment.
- All state-changing forms use POST and Django CSRF tokens.
- Uploaded files are limited to 5 MB, decoded with Pillow, restricted to JPEG, PNG, or WebP, and renamed with random safe names.
- Reports do not export passwords, email addresses, phone numbers, or authentication data.
- CSV cells beginning with =, +, -, or @ are prefixed with an apostrophe before export.

Chapter 5 Implementation

5.1 Technology Stack

Technology	Version / use	Reason
Python	Application language	Readable ecosystem and strong Django support
Django	6.1.1	Authentication, ORM, forms, templates, testing and admin
SQLite	Local database	Zero-configuration single-file academic deployment
Bootstrap	5.3.8 via CDN	Responsive navigation, forms and grid
Chart.js	4.4.7 via CDN	Administrator analytics charts
Pillow	11.3.0	Decode and validate uploaded images

5.2 Authentication and Authorisation

CampusFix extends Django's AbstractUser with a STUDENT, STAFF, or ADMIN role and institutional profile fields. Public registration always assigns STUDENT regardless of submitted extra fields. A dashboard router sends authenticated users to the correct dashboard. A reusable role_required decorator returns a branded HTTP 403 page for cross-role access.

5.3 Complaint Submission and Images

The ComplaintForm exposes only student-controlled fields. The view assigns the logged-in Student and Submitted status. The model creates ticket numbers in the form CF-year-sequence. Image validation checks size, content, and detected format rather than trusting only the file extension.

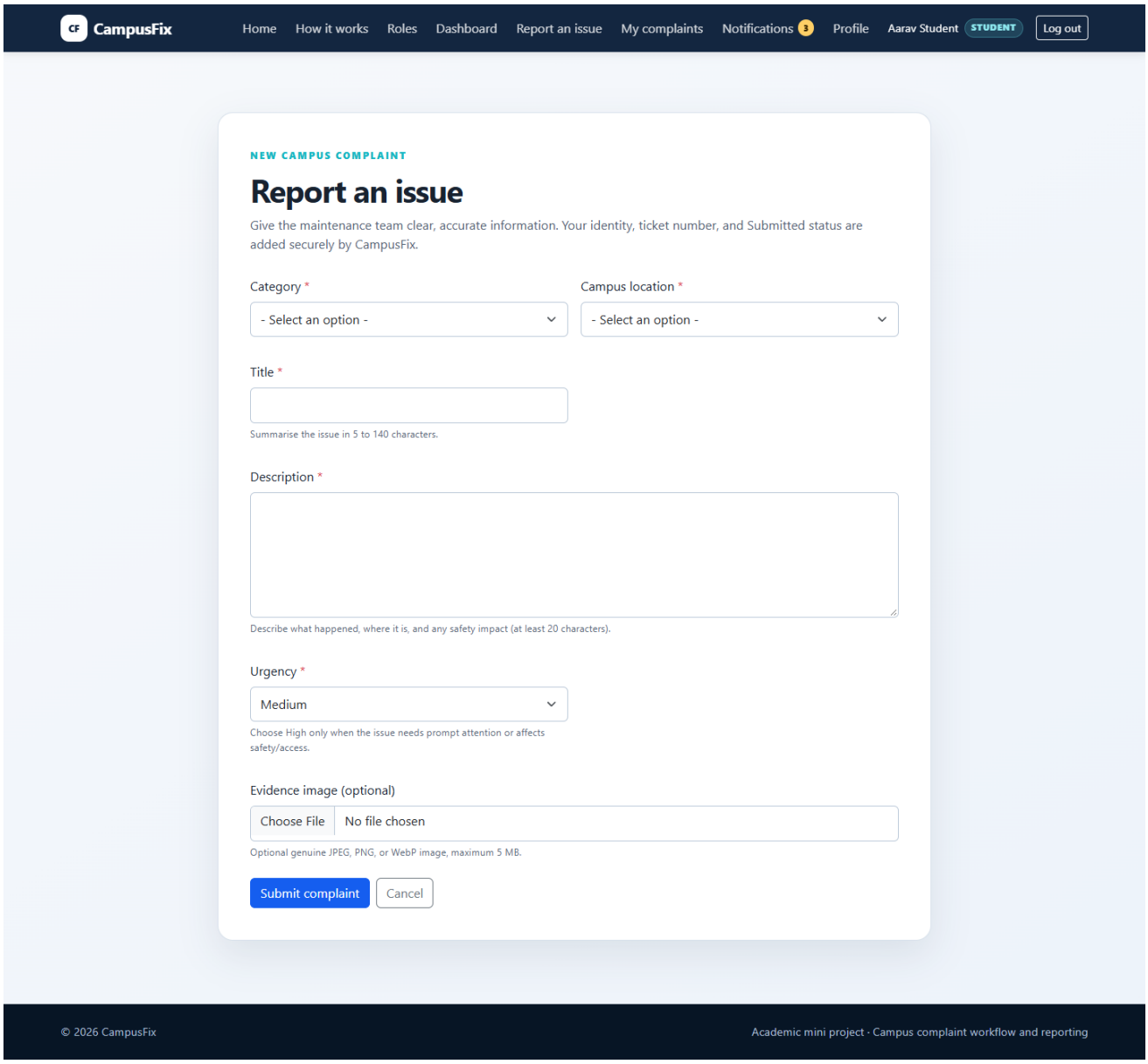


Figure 8. Student complaint submission form

5.4 Workflow and Timeline

Workflow rules are centralised in service functions using database transactions. Administrators and assigned Staff can perform only transitions valid for the current state. Each successful transition appends ComplaintHistory with previous status, new status, actor, note, and timestamp.

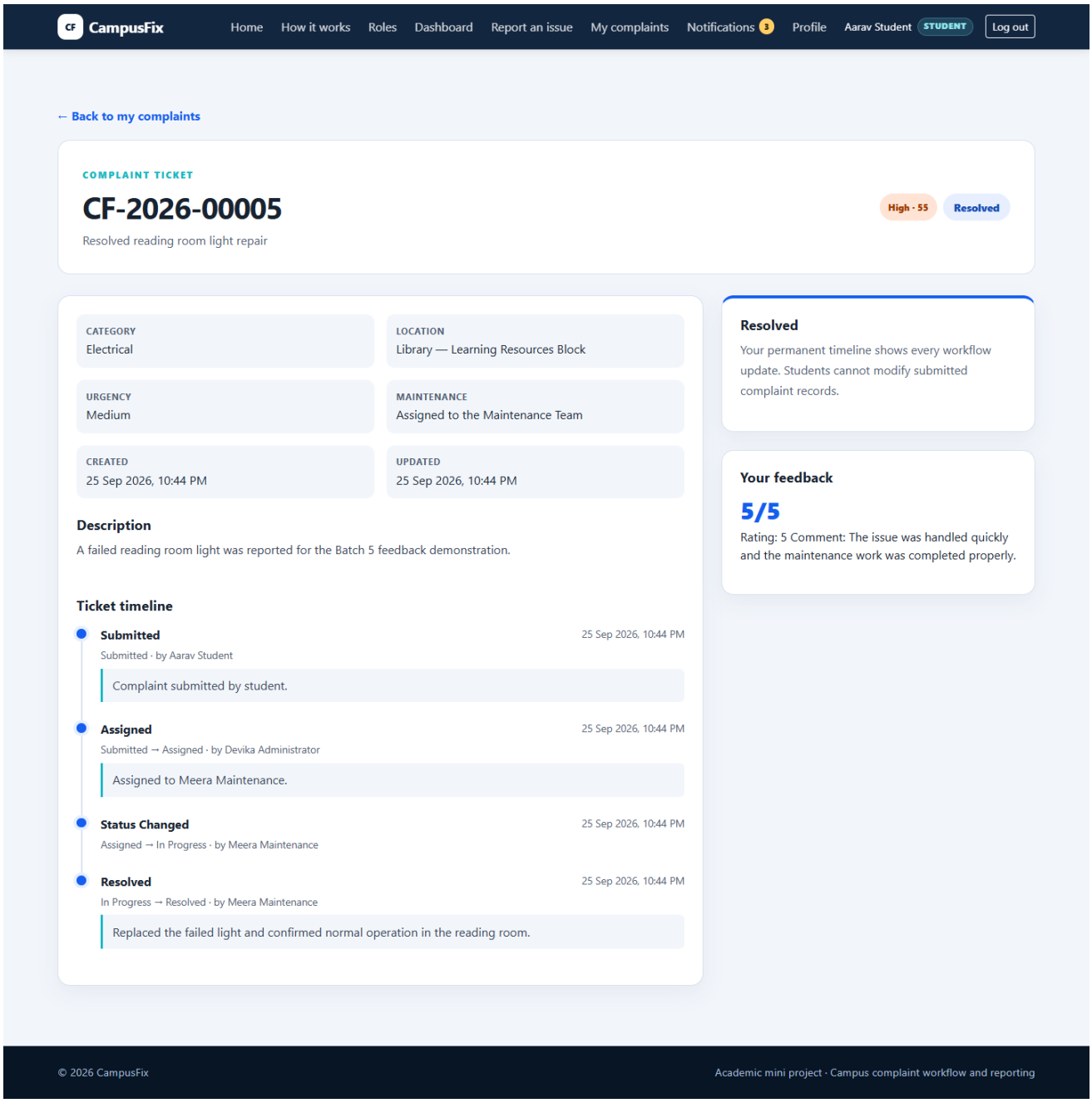


Figure 9. Student ticket detail with priority, feedback, and timeline

5.5 Notifications and Feedback

Notifications are written to specific recipients with deduplication keys. Users see unread counts, can open individual notifications, and can mark all as read through a CSRF-protected POST. A Student may submit one rating from 1 to 5 and an optional comment only after the complaint is Resolved or Closed.

5.6 Analytics and Reporting

Administrator analytics aggregate complaints by status, category, priority, urgency, and period. Reports add custom dates, staff and search filters, live totals, a workload table, feedback rows, pagination, three UTF-8

CSV downloads, and a print view. Related records are selected in the main query to avoid repeated per-row lookups.

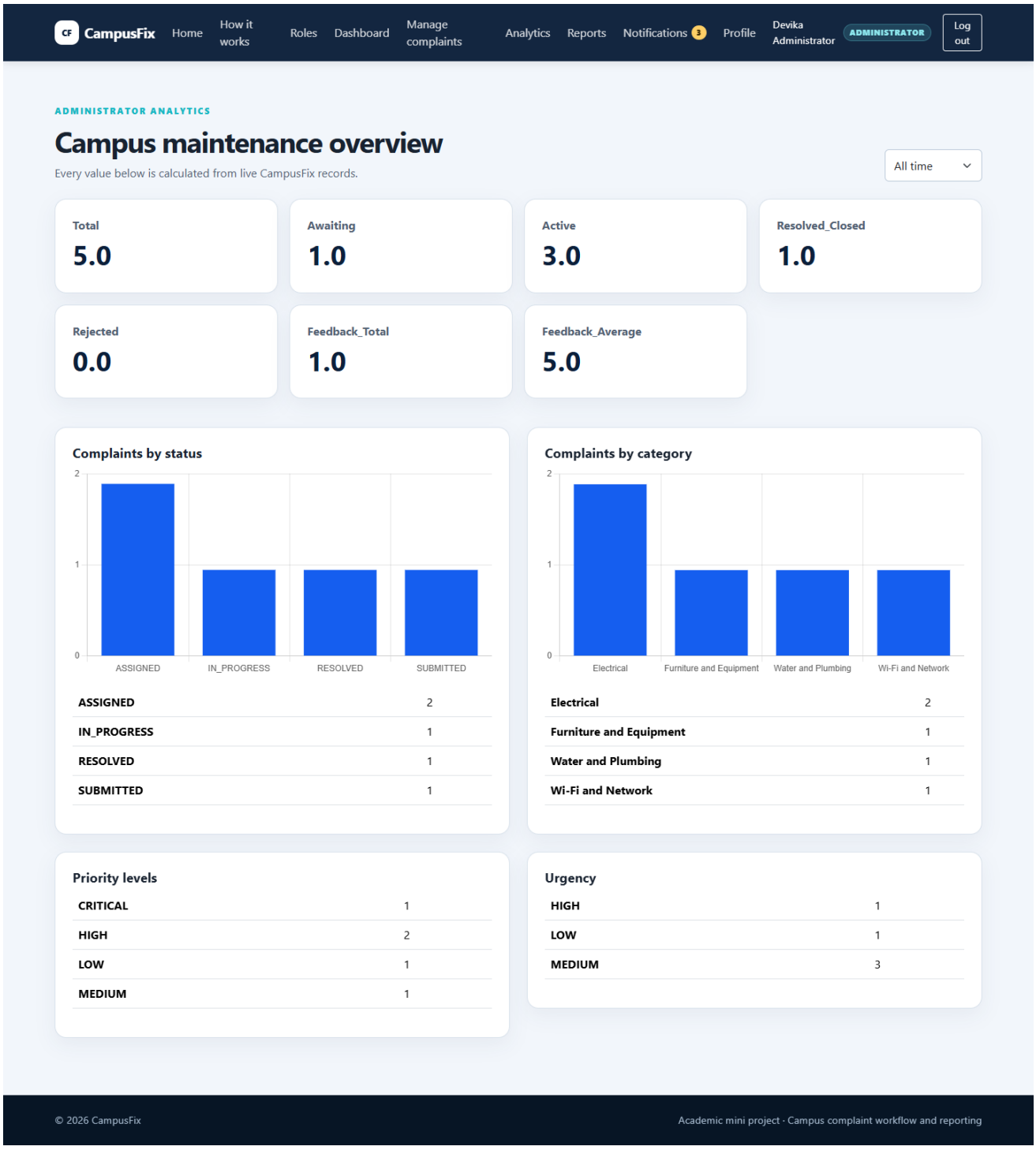


Figure 12. Administrator analytics with live charts

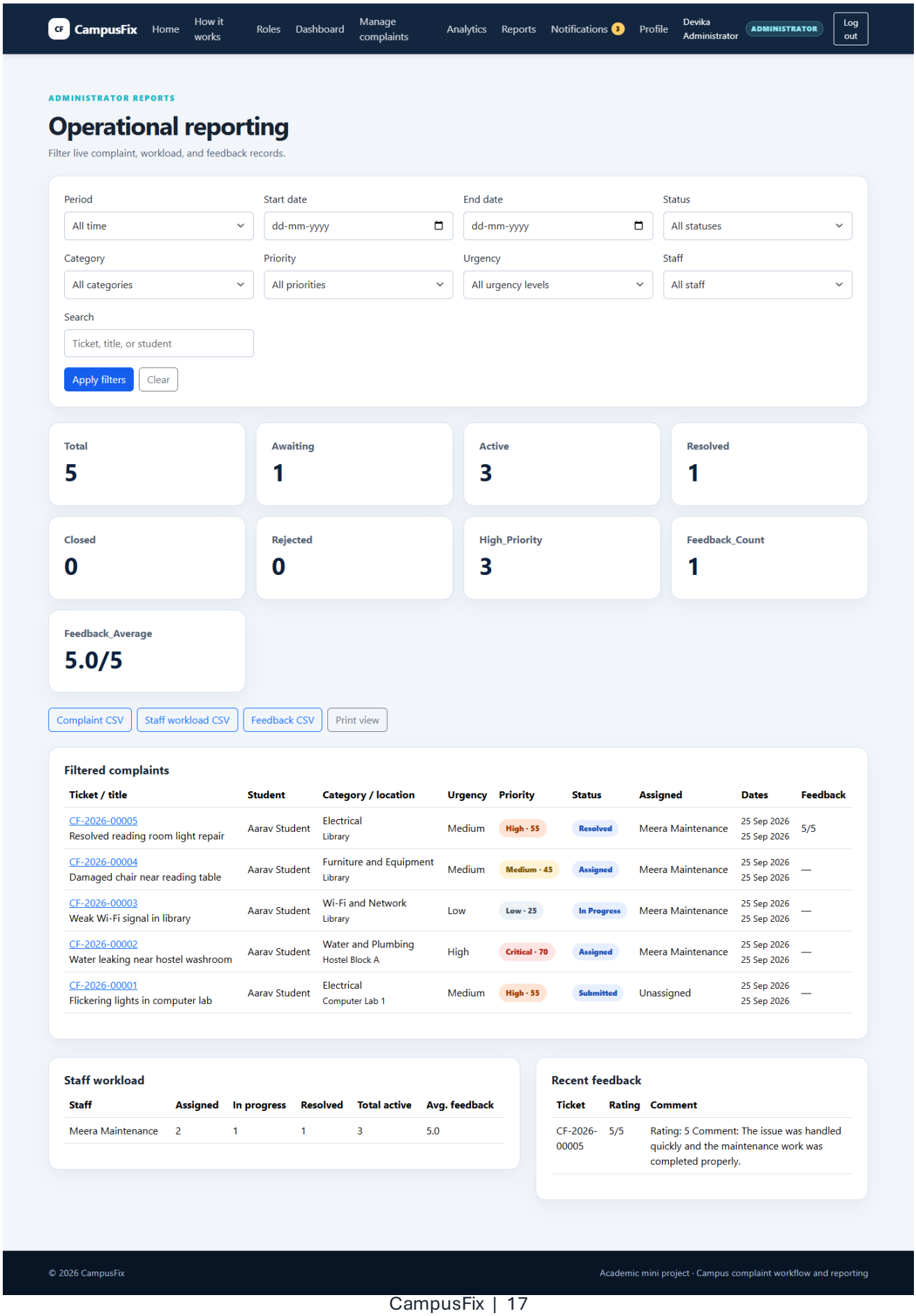

CampusFix
Home
How it works
Roles
Dashboard
Manage complaints
Analytics
Reports
Notifications 3
Profile
Devika Administrator
ADMINISTRATOR
Log out
ADMINISTRATOR REPORTS
Operational reporting
Filter live complaint, workload, and feedback records.
Period
All time
Start date
dd-mm-yyyy
End date
dd-mm-yyyy
Status
All statuses
Category
All categories
Priority
All priorities
Urgency
All urgency levels
Staff
All staff
Search
Ticket, title, or student
Apply filters
Clear
Total
5
Awaiting
1
Active
3
Resolved
1
Closed
0
Rejected
0
High_Priority
3
Feedback_Count
1
Feedback_Average
5.0/5
Complaint CSV
Staff workload CSV
Feedback CSV
Print view
Filtered complaints
Ticket / title
Student
Category / location
Urgency
Priority
Status
Assigned
Dates
Feedback
CF-2026-00005
Resolved reading room light repair
Aarav Student
Electrical
Library
Medium
High · 55
Resolved
Meera Maintenance
25 Sep 2026
25 Sep 2026
5/5
CF-2026-00004
Damaged chair near reading table
Aarav Student
Furniture and Equipment
Library
Medium
Medium · 45
Assigned
Meera Maintenance
25 Sep 2026
25 Sep 2026
—
CF-2026-00003
Weak Wi-Fi signal in library
Aarav Student
Wi-Fi and Network
Library
Low
Low · 25
In Progress
Meera Maintenance
25 Sep 2026
25 Sep 2026
—
CF-2026-00002
Water leaking near hostel washroom
Aarav Student
Water and Plumbing
Hostel Block A
High
Critical · 70
Assigned
Meera Maintenance
25 Sep 2026
25 Sep 2026
—
CF-2026-00001
Flickering lights in computer lab
Aarav Student
Electrical
Computer Lab 1
Medium
High · 55
Submitted
Unassigned
25 Sep 2026
25 Sep 2026
—
Staff workload
Staff
Assigned
In progress
Resolved
Total active
Avg. feedback
Meera Maintenance
2
1
1
3
5.0
Recent feedback
Ticket
Rating
Comment
CF-2026-00005
5/5
Rating: 5 Comment: The issue was handled quickly and the maintenance work was completed properly.
© 2026 CampusFix
Academic mini project · Campus complaint workflow and reporting

Figure 13. Administrator filtered reports and exports

Chapter 6 Testing and Results

6.1 Testing Approach

Testing combines unit tests for models and forms, permission tests for all roles, workflow service tests, page and ownership checks, demo-command idempotence tests, export-security tests, Django configuration checks, migration checks, static collection, and live HTTP smoke checks. Test data is created in an isolated temporary database.

6.2 Automated Evidence

Verification	Result
Django system check	0 issues
Missing migration check	No changes detected
Database migration	No pending migrations
Static collection	Successful
Full automated suite	84 tests passed in 131.552 seconds
Live report checks	Administrator 200; Student and Staff 403
CSV response checks	200 with text/csv; charset=utf-8

6.3 Working Demonstration Evidence

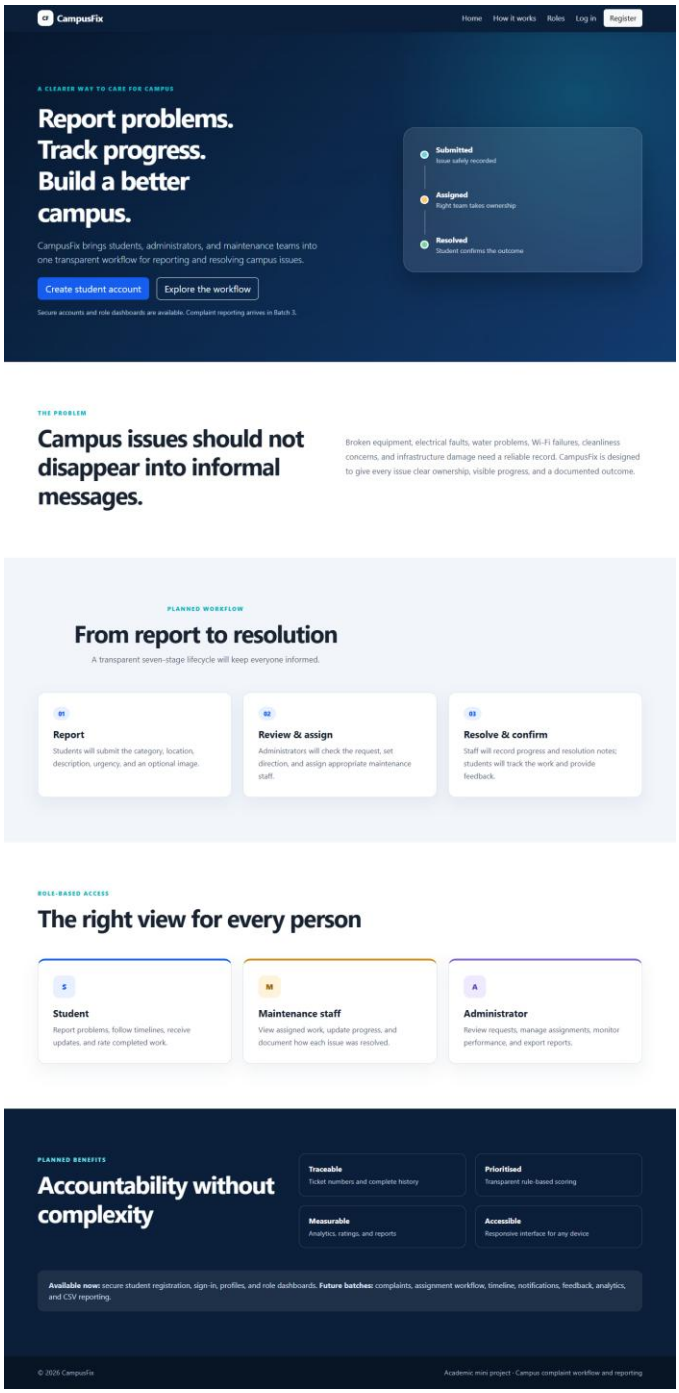


Figure 6. Public CampusFix landing page

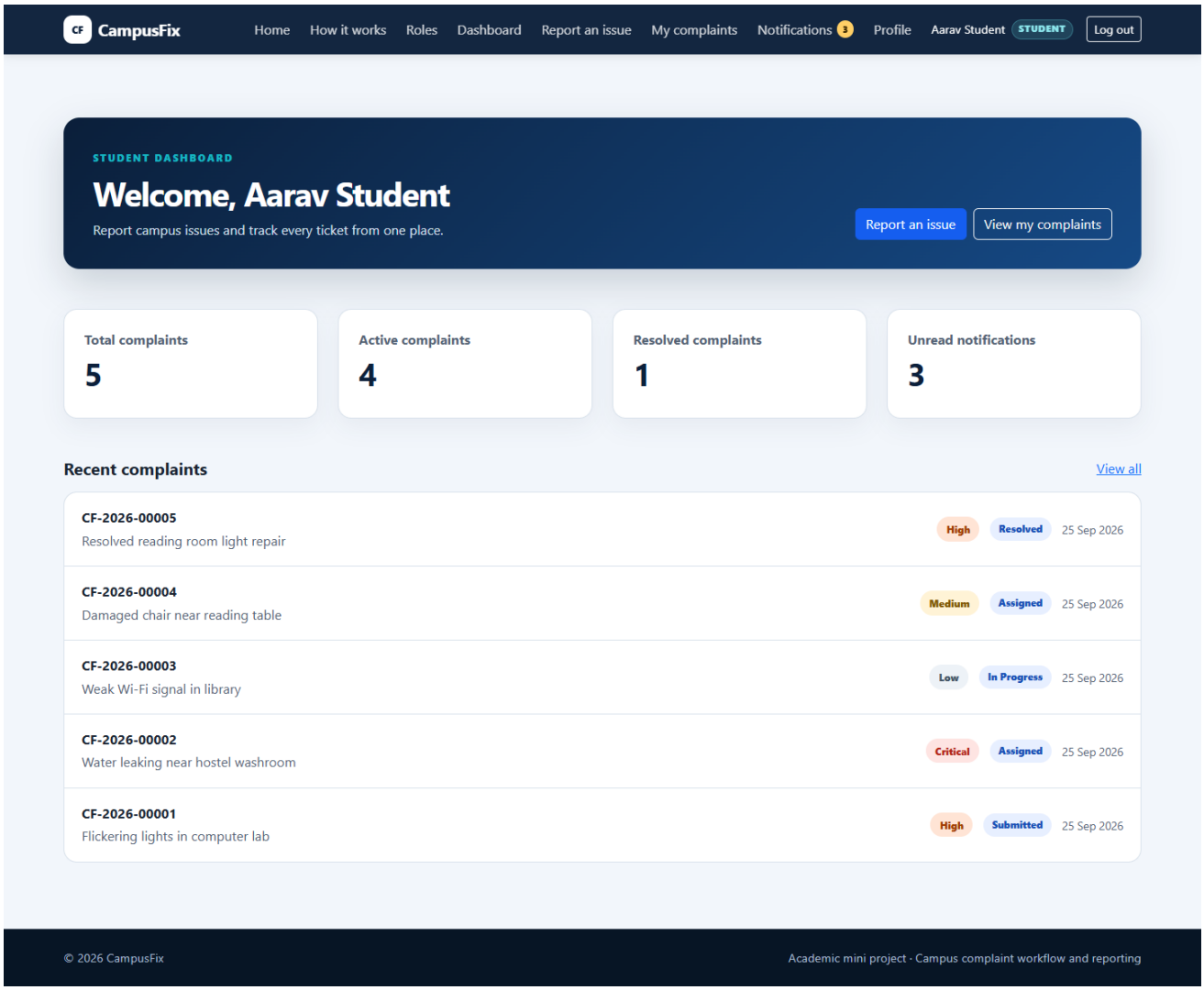


Figure 7. Student dashboard with live ticket counts

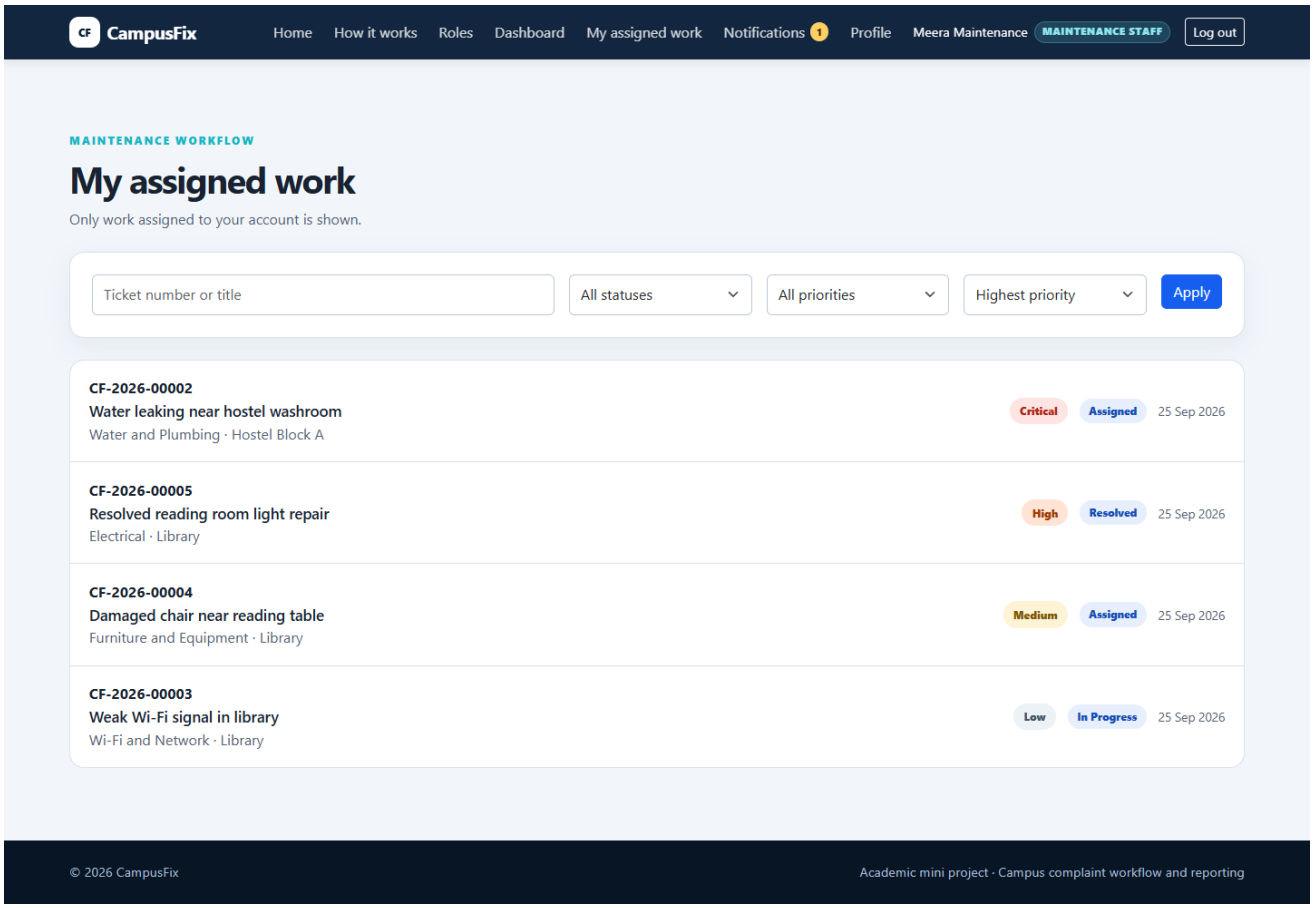


Figure 10. Maintenance Staff assigned-work queue

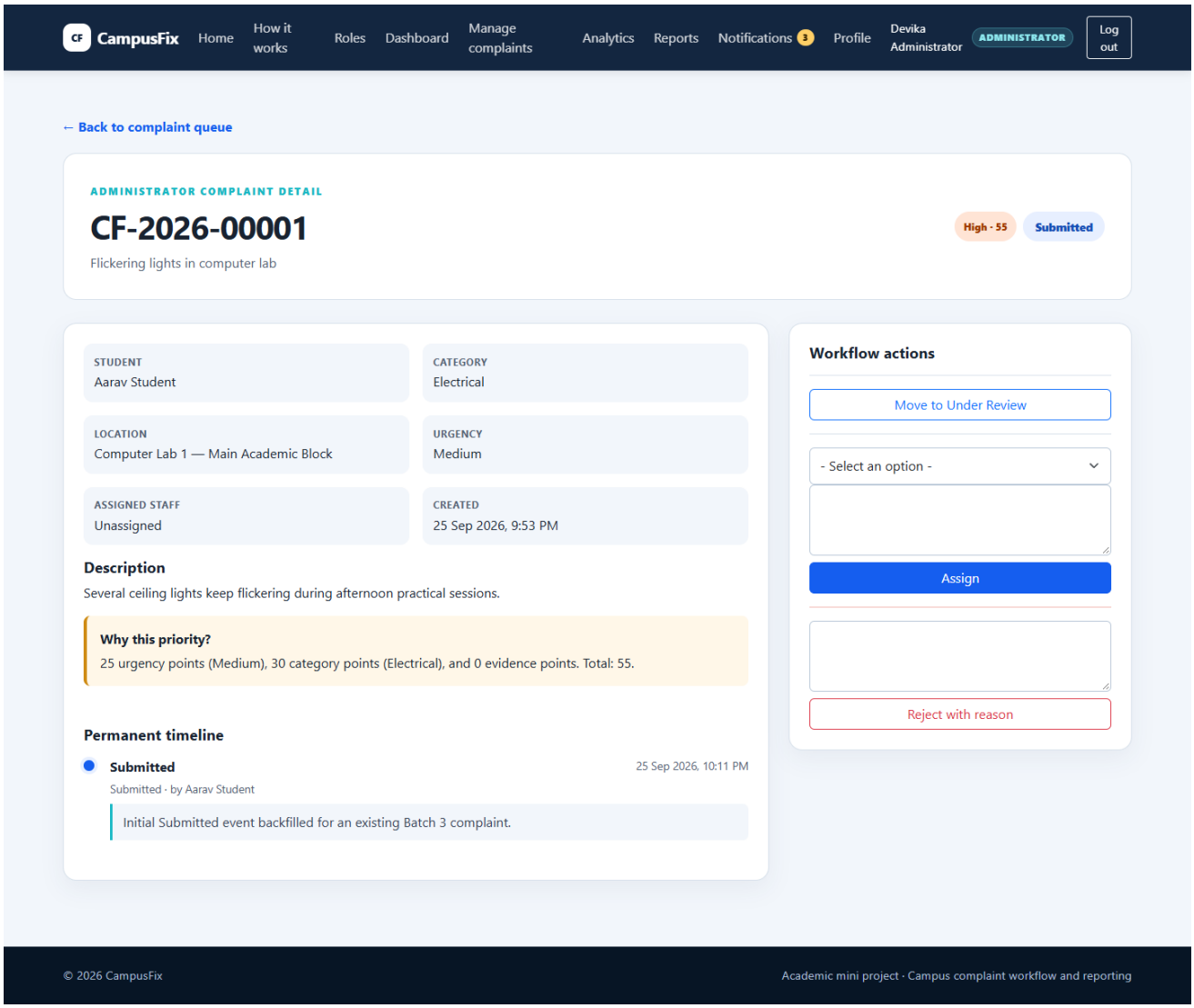


Figure 11. Administrator complaint detail and assignment controls

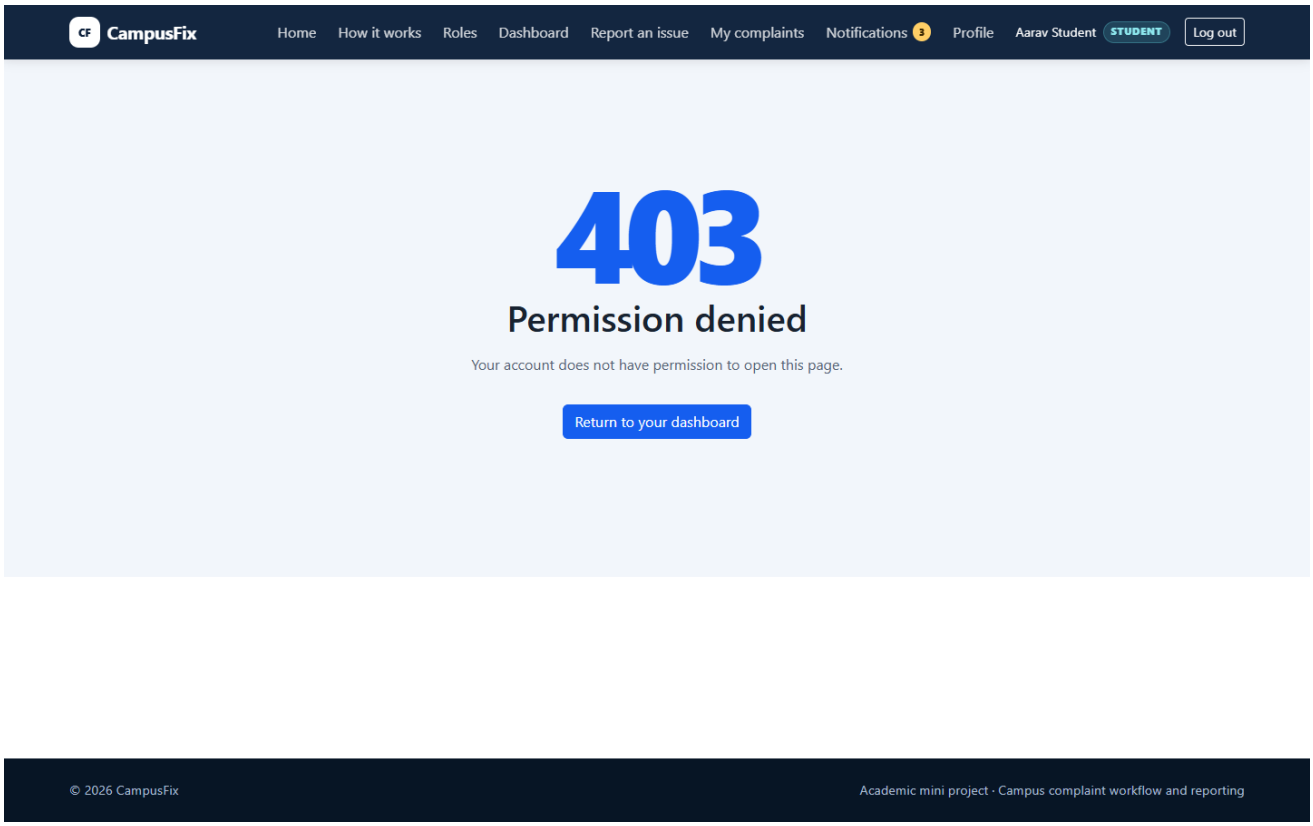


Figure 14. Branded server-side permission denial

6.4 Key Results

- A complaint remains traceable from submission through resolution and closure.
- Students cannot read another Student's ticket; Staff cannot act on unassigned work.
- Administrators can see live workload, priority, feedback, analytics, and filtered exports.
- Demo data commands can be rerun without duplicating their reference records.
- All accepted Batches 1 through 6 remain covered by the passing regression suite.

Chapter 7 Conclusion and Future Scope

7.1 Conclusion

CampusFix meets the project objective of replacing fragmented campus maintenance reporting with a controlled and transparent workflow. Its strongest result is the connection between role-based permissions, explicit status transitions, a permanent timeline, and evidence-based administrator reporting. The completed application is suitable for a fourth-year mini-project demonstration and provides clear opportunities for deployment-oriented extension.

7.2 Future Scope

- Email and SMS delivery for important status changes.
- A native or progressive mobile application.
- Production deployment with PostgreSQL, managed media storage, HTTPS, monitoring, and backups.

- Predictive maintenance using historical recurrence and resolution-time data.
- Richer service-level and response-time analytics.
- A multilingual interface and accessibility testing with assistive technologies.

References

[1] R. Sandhu, D. Ferraiolo, and D. R. Kuhn, The NIST Model for Role-Based Access Control: Towards a Unified Standard, ACM RBAC 2000. https://doi.org/10.1145/344287.344301

[2] Django Software Foundation, Using the Django authentication system, Django 6.1 documentation. https://docs.djangoproject.com/en/6.1/topics/auth/default/

[3] Django Software Foundation, Security in Django, Django 6.1 documentation. https://docs.djangoproject.com/en/6.1/topics/security/

[4] Atlassian, What are Jira workflows? https://www.atlassian.com/software/jira/guides/workflows/overview

[5] SQLite Consortium, About SQLite. https://www.sqlite.org/about.html

[6] Chart.js, Accessibility. https://www.chartjs.org/docs/latest/general/accessibility.html

[7] OWASP Foundation, CSV Injection. https://community.owasp.org/attacks/CSV_Injection

[8] Bootstrap Team, Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

[9] Django Software Foundation, File uploads, Django 6.1 documentation. https://docs.djangoproject.com/en/6.1/topics/http/file-uploads/

[10] Pillow Contributors, Pillow Documentation. https://pillow.readthedocs.io/en/stable/

Appendix A Important Routes

Route	Purpose
/	Public landing page
/register/ and /login/	Student registration and authentication
/dashboard/	Role dashboard router
/complaints/new/	Student complaint submission
/complaints/	Student ticket list
/work/	Maintenance Staff queue
/management/complaints/	Administrator complaint queue
/analytics/	Administrator analytics
/reports/	Administrator reports
/admin/	Django Admin for authorised superusers

Appendix B Local Demo Accounts

LOCAL DEMO ONLY. These credentials are created only by the DEBUG-only demo command and must never be used in production.

Role	Username	Password
Student	student_demo	CampusFixStudent@2026
Maintenance Staff	staff_demo	CampusFixStaff@2026
Administrator	admin_demo	CampusFixAdmin@2026

Appendix C Priority Scoring

Component	Points
Urgency: Low / Medium / High	10 / 25 / 40
Safety	40
Electrical or Water	30
Furniture	20
Wi-Fi	15
Cleanliness or Other	10
Valid image evidence	+5
Level thresholds	Low <30; Medium 30-49; High 50-69; Critical >=70

Appendix D CSV Safety Note

Spreadsheet software may interpret cells beginning with =, +, -, or @ as formulas. CampusFix converts nulls to blank text and prefixes those leading characters with an apostrophe before writing CSV rows. The files also begin with a UTF-8 byte-order mark for practical spreadsheet compatibility. No universal CSV strategy is safe for every spreadsheet and downstream import, so exports should still be treated as data files from a trusted local administrator workflow.